SIMATS ENGINEERING

Department of Information Technology

NATURAL LANGUAGE PROCESSING

ASSIGNMENT 5 – CO5

LANGUAGE GENERATION AND DISCOURSE ANALYSIS

CO5: Discourse and reference resolution – text coherence and discourse structure – dialog and conversational agents – dialog acts and interpretation – language generation – discourse planning and surface realization – machine translation.

Student Name: G. Sirisha

Register Number: 192472170

Department / Section: CSE-AI

Faculty Name: Premkumar R

Date of Submission: 06-11-26

PART A – DESIGN, SPECIFICATION & PSEUDOCODE

Common Text T

The given text is:

1. The department bought a new server for the lab last month.
2. It replaced an old machine that had failed twice.
3. Ravi configured the server with the lab's toolchain.
4. He then installed the compiler suite on it.
5. The suite crashed when the memory filled during a build.
6. Ravi reported the fault to the vendor, and the vendor replaced a faulty module.

Gold reference links:

- **It → server** in sentence 2
- **He → Ravi** in sentence 4
- **it → server** in sentence 4
- **The suite → compiler suite** in sentence 5

A1. DEFINE THE DISCOURSE ANALYST'S WORLD

A1.1 Position and Responsibilities

Discourse Module

The discourse module is responsible for understanding how sentences are connected.

Input:

- Sentences
- Clauses
- Named entities
- Pronouns
- Previous discourse context

Output:

- Discourse units
- Reference links
- Entity chains
- Discourse relations
- Salience information

It should **preserve the original meaning** and should not arbitrarily change facts.

Generation Module

The generation module takes the semantic/discourse representation and produces natural language.

Input:

- Meaning representation
- Discourse structure
- Entity information
- Focus information

Output:

- Referring expressions
- Sentence order
- Surface text

It may change wording and sentence structure, but it must **preserve the intended meaning**.

Issue	**Example**	**Main decision**
Unit of analysis	Sentence, clause or turn	Annotation scheme
Knowledge source	Agreement, syntax, salience, world knowledge	System designer
Explicit/implicit relation	Therefore indicates Result	Annotation + designer
Discourse granularity	Sentence-level or clause-level	Annotation scheme
Evaluation	Precision, Recall, F1	System designer
Genuine ambiguity	He may have two antecedents	System designer

A1.2 Measurement Model

Markable

A **markable** is a referring expression that can potentially refer to an entity.

Examples:

- “It”
- “an old machine”

- “Ravi”
- “He”
- “the server”
- “the suite”

Correct Resolution

A resolution is correct when the predicted antecedent is the same as the gold antecedent.

The four gold links are:

1. It → server
2. He → Ravi
3. it → server
4. The suite → compiler suite

System	Correct	Predicted	Gold	Precision	Recall	F1
X	3	5	4	60%	75%	66.67%
Y	3	3	4	100%	75%	85.71%

Limitations

This measure does not fully capture:

- Bridging references
- Near-identity relations
- Effects of one wrong reference on the final answer
- Discourse coherence
- Whether the final generated answer is correct

Therefore, a system may have a good F1 score but still produce a wrong final answer. The assignment specifically warns that a plausible final answer without reasoning and evidence is insufficient.

A1.3 Reference Resolution Procedure

ALGORITHM ResolvePronoun(P, Context)

INPUT:

Pronoun P

Previous discourse entities

OUTPUT:

Antecedent or UNRESOLVED

BEGIN

1. If P is pleonastic/non-referential

Return NON_REFERENTIAL

2. Create candidate set C from previous entities.

3. Remove candidates that fail:

a. Number agreement

b. Person agreement

c. Syntactic compatibility

4. Score remaining candidates using:

a. Recency

b. Grammatical role

c. Repetition

d. Centering/salience

5. Sort candidates by score.

6. If C is empty

Return UNRESOLVED

7. If the highest candidate has a unique score

Return highest candidate

8. If two candidates tie

Return AMBIGUOUS

9. Otherwise

Return highest-scoring candidate

END

Salience Order

$$Subject > Topic > Object > Recent\ Mention$$

The resolver therefore does not simply choose the nearest noun. It combines agreement, syntactic role, recency and repetition, as required by the assignment.

A1.4 Trace the Resolver

Sentence 1

“The department bought a new server for the lab last month.”

Entities:

- department
- server
- lab

No pronoun.

State:

Focus = server > department > lab

Sentence 2

“It replaced an old machine that had failed twice.”

Pronoun:

It

Candidate set:

Candidate	Agreement	Role	Recency	Score
server	Yes	High	High	0.90
department	Yes	Low	Medium	0.35
lab	Yes	Low	Medium	0.25

Chosen:

It → server

State:

server

old machine

department

lab

Sentence 3

“Ravi configured the server with the lab's toolchain.”

New entities:

- Ravi
- toolchain

The existing server is mentioned again.

State:

Ravi > server > toolchain

Sentence 4

“He then installed the compiler suite on it.”

Pronoun 1: He

Candidates:

Candidate	**Agreement**	**Subject role**	**Recency**	**Score**
Ravi	Yes	Very high	High	0.95
server	No	Object	High	0
toolchain	No/weak	Object	Medium	0

Therefore:

He → Ravi

Pronoun 2: it

Candidates:

Candidate	**Compatibility**	**Role**	**Score**
server	Yes	Previous object/topic	0.90
compiler suite	Not yet established	—	0

Therefore:

it → server

Simplistic Rule That Can Fail

A simple **nearest-noun rule** may choose the wrong antecedent because the closest noun is not always the discourse topic.

For example, in sentence 4, the correct antecedent of **He** is Ravi, even though other entities occur in the previous sentence.

Our design avoids this by using:

agreement + grammatical role + salience + recency.

A2. FIND THE STRUCTURE HIDDEN IN A FLAT LIST

A2.1 Discourse Boundaries

A new discourse unit is created when there is a significant change in:

1. Main topic
2. Event
3. Time
4. Location
5. Discourse relation
6. Entity focus

Units for Text T

A reasonable segmentation is:

- **U1:** Sentence 1
- **U2:** Sentence 2
- **U3:** Sentences 3–4
- **U4:** Sentence 5
- **U5:** Sentence 6

Interpretation

U1 → Introduction of server

U2 → Background/history of server

U3 → Ravi configures and installs software

U4 → Software failure

U5 → Reporting and vendor response

A2.2 Segmentation and Discourse Graph

Pseudocode

```
ALGORITHM BuildDiscourseGraph(Text)
INPUT:
   Text containing sentences
OUTPUT:
   Discourse graph G
BEGIN
1. Split Text into sentences.
2. For each sentence:
     Detect topic, entities, time and cue phrases.
3. Create a discourse unit for each major event/topic.
4. For every adjacent pair:
     IF explicit cue phrase exists
        assign corresponding relation
     ELSE IF second unit explains first
        assign EXPLANATION
     ELSE IF second unit describes result
        assign RESULT
     ELSE IF second continues the same event
        assign ELABORATION
     ELSE
        assign SEQUENCE
5. Store:
```

relation

nucleus

satellite

predecessor

successor

6. Check right-frontier constraint.

7. Return graph.

END

Discourse Graph

U1: Department bought server

|

| ELABORATION

v

U2: Server replaced old machine

|

| SEQUENCE

v

U3: Ravi configured and installed software

|

| RESULT

v

U4: Compiler suite crashed

|

| EXPLANATION / SEQUENCE

v

U5: Ravi reported fault → Vendor replaced module

The deepest attachment is around **U4**, because the failure is the consequence of the software installation and memory condition.

The structural constraint is the **Right Frontier Constraint**: a new discourse unit cannot arbitrarily attach to a unit that is already closed.

A2.3 Next-Mention Information

A backward scan is preferred because the next occurrence of an entity can be found by scanning from the end toward the beginning.

Example

Unit	Mention	Entity	Next mention	Distance
U2	It	server	U3	1
U2	old machine	old machine	None	—
U3	Ravi	Ravi	U5	2
U3	server	server	U4/U5	1
U3	toolchain	toolchain	None	—

Exit Assumption

Entities on the **right frontier** remain available for future discourse.

Entities that never appear again can be removed from active memory.

A2.4 Structural Pitfalls

1. Cue phrase in the middle

A phrase such as **"when"** inside a sentence should not automatically create a new discourse unit.

If every connective created a boundary, one sentence could be incorrectly split into several unrelated units.

2. Forward next-mention scan

A forward scan does not naturally know whether a current entity will appear again.

A backward scan can directly identify the nearest later mention.

3. Why a summariser needs the graph

A graph identifies:

- Main events
- Causes
- Results
- Topic shifts
- Important versus supporting information

A flat list only gives sentence order.

A3. DESIGN A GENERATOR THAT REMEMBERS WHAT IT HAS ALREADY SAID

Microplan:

F1 memory(server_1, 32 GB)

F2 configure(ravi, server_1)

F3 install(ravi, suite_1, server_1)

F4 crash(suite_1, during=build)

Focus size k = 2. A full name/definite description costs 3 words and a pronoun costs 1 word.

A3.1 Required Information

- Focus list: up to two currently salient entities.
- Realisation descriptor: last form used, number and gender.
- Aggregation buffer: propositions that may be combined.
- Given/new flag: whether an entity has already been introduced.

A3.2 Referring-Expression Choice

ALGORITHM CHOOSE_FORM(Entity E)

IF E is in focus AND no same-gender competitor exists

  RETURN PRONOUN

ELSE IF a focus slot is free

RETURN NAME or DEFINITE_DESCRIPTION

ELSE

Evict least salient entity

RETURN DEFINITE_DESCRIPTION

IF E is new

RETURN INDEFINITE_NOUN_PHRASE

END IF

Selection rule: use the shortest form that uniquely identifies the entity without creating ambiguity.

IF two propositions have the same subject and compatible order/tense

THEN aggregate using "and"

ELSE generate separate sentences.

A3.3 Trace the Generator

Step	Generated text	Focus	Approx. words
F1	The server has 32 GB of memory.	server	7
F2	Ravi configured the server.	Ravi, server	5
F3	Ravi configured the server and installed the compiler suite on it.	compiler suite, server	11
F4	It crashed during the build.	compiler suite	5

The generator reduces repetition by reusing focus entities and aggregating compatible propositions.

A3.4 Failure Case

Consider:

F1 = Ravi entered the lab.

F2 = Ravi spoke to Arun.

F3 = Arun gave Ravi the report.

F4 = He left.

The pronoun **He** is ambiguous because both Ravi and Arun are male.

A generator that only uses recency may produce:

“He left.”

A human may instead write:

“Ravi left the lab.”

Missing Information

The generator needs:

- Hearer model
- Centering transition
- Paragraph context
- Entity importance
- Ambiguity detection

Therefore, short referring expressions are not always better.

A4. IMPROVE A WEAK DISCOURSE PARSER

The weak algorithm assumes:

- Shared noun → ELABORATION
- No connective → SEQUENCE

This is unreliable because lexical overlap does not guarantee the same discourse relation.

A4.1 Diagnose the Defect

1. Ravi configured the lab server on Monday.
2. It crashed during the first build.
3. A second server arrived from the vendor on Tuesday.
4. Ravi configured it as well.
5. The room had become very warm by then.

Pair	Weak parser	Better interpretation
1→2	SEQUENCE	RESULT / CONSEQUENCE
3→4	ELABORATION	Elaboration / Sequence about Server 2
4→5	SEQUENCE	Possible explanation / cause

Shared nouns and absence of connectives do not uniquely determine discourse relations.

A4.2 Entity–Discourse Graph

Each real-world entity gets exactly one node.

Example:

Ravi ----------> Ravi_1

"Ravi" in S1/S4

Server 1 ----------> Server_1

"lab server" S1

"It" S2

Server 2 ----------> Server_2

"second server" S3

"it" S4

Room ----------> Room_1

"room" S5

Pseudocode

```
ALGORITHM BuildEntityGraph(Text)

FOR each sentence S:
    Detect noun phrases and pronouns
    FOR each mention M:
        IF M matches an existing entity
            Link M to existing node
        ELSE IF M is a new description
            Create new entity node
        ELSE IF M is a pronoun
            Generate compatible candidates
            Resolve using salience + agreement
        END IF
    Detect discourse relation
    Attach current unit to the appropriate discourse node
END FOR

RETURN EntityGraph
```

A4.3 Demonstrate the Improvement

- Same noun, different entity: 'The lab bought a server. A second server arrived.' → Server_1 and Server_2.
- Cataphora: 'When he arrived, Ravi started the meeting.' → he = Ravi.
- One-anaphora: 'The first server failed. A faster one replaced it.' → one refers to a server entity/type.
- Implicit causal relation: 'The room became very warm. The server failed.' → possible causal relation, not guaranteed fact.

IF a full name has already been introduced AND the entity is uniquely identifiable

THEN use a shorter referring expression.

IF a sentence repeats a proposition already generated

THEN suppress the repeated sentence.

A4.4 Challenge

Reported Speech

Quoted speech can create a separate discourse context.

Example:

Ravi said, “It failed.”

The **it** may refer to an entity inside Ravi's statement rather than the main document focus.

Section Heading

A heading such as:

“Vendor Response”

may start a new discourse segment.

Old entities may still exist globally but may lose local salience.

Machine Translation

Gender may disappear during translation.

For example, an English **“he/she”** distinction may become a gender-neutral pronoun.

Violated Assumption

The original design assumes:

mention form + grammatical features are reliable indicators of entity identity.

Machine translation can violate this assumption.

A5. DISCOVER WHAT THE WHOLE DOCUMENT KNOWS

A5.1 Opportunities

Technique	Opportunity	Safety condition
Coreference chains	Consistent entity tracking	Resolution must be reliable
Lexical chaining	Topic identification	Avoid false chains
Centering	Pronoun/name choice	Local salience must be accurate
Entity grid	Coherence scoring	Entity roles must be correct
Discourse structure	Summarisation	Relations must be justified
Distributional similarity	Related concepts	Similarity is not factual identity

A5.2 Global Computation

GEN

Entities newly introduced by a unit.

Example:

U3:

GEN = {Ravi, compiler_suite, toolchain}

KILL

Entities that are no longer active or whose salience is replaced.

Example:

KILL = {old_machine}

Pseudocode

ALGORITHM GlobalSalience(Graph)

INITIALIZE:

    Entry[U1] = {}

    Change = TRUE

WHILE Change = TRUE:

```
  Change = FALSE
  FOR each unit U in document order:
    Entry[U] =
      Union of predecessor Exit sets
    NewExit =
      (Entry[U] - KILL[U]) UNION GEN[U]
    IF NewExit != Exit[U]
      Exit[U] = NewExit
      Change = TRUE
END WHILE
RETURN Entry, Exit
```

Why It Terminates

The number of discourse entities is finite.

Each iteration either:

- adds an entity, or
- removes/replaces an entity.

Eventually no set changes.

Therefore, the computation reaches a **fixpoint**.

A5.3 Trace

Unit	Entry	Generated	Killed	Exit
U1	—	E1 server	—	E1
U2	E1	E2 old machine	—	E1,E2
U3	E1,E2	E3 Ravi; E4 toolchain; E5 suite	E2	E1,E3,E4,E5
U4	E1,E3,E4,E5	—	E4	E1,E3,E5

U5	E1,E3,E5	E6 vendor; E7 module	—	E1,E3,E5,E6,E7

A5.4 Backward Analysis

To determine what the reader **must already know**, propagation works backward.

Instead of asking:

“What information becomes available after this sentence?”

we ask:

“What information must already be available before this sentence?”

Direction

Backward

Meet Operation

When multiple paths meet:

Intersection is safer than union.

Initialisation

Start from the required information at the end and propagate backwards.

Why Conservative?

Intersection keeps only information that is guaranteed on **all paths**.

Therefore, the result may omit information that is actually known from a particular path.

Effect on Applications

A summariser may lose some useful background information.

A translator may lose contextual assumptions.

However, the conservative result is safer because it avoids claiming that the reader knows something that has not been established.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

Required workflow: Implement → Instrument → Stress → Verify → Measure → Compare → Interpret.

B1. IMPLEMENT AND INSTRUMENT THE DISCOURSE ANALYSER

B1.1 Test Cases

Test 1 – Topic Shift

The department installed a new server.

Ravi configured the server.

The library received new books.

Students borrowed the books.

Purpose:

- Detect topic shift
- Verify segmentation
- Test whether old entities remain salient

Test 2 – Same Head Noun

The lab bought a server.

A second server arrived.

It was faster than the first server.

Purpose:

- Verify that two occurrences of **server** can represent different entities.
- Test pronoun resolution.

Test 3 – Boundary Case

The server crashed.

Purpose:

- Test a one-sentence input.
- Ensure the system does not generate unnecessary discourse relations.

B1.2 Design vs Implementation

Modifications

During implementation, the following modifications may be required:

1. Sentence splitting must handle punctuation.
2. Pronoun filtering must remove non-referential **it**.
3. Paragraph boundaries may be treated as strong discourse boundaries.
4. Candidate scoring must handle ties.

These modifications make the practical system more robust while preserving the original design.

B1.3 Two Resolution Strategies

Version 1 – Most Recent Compatible

Choose the most recent candidate that satisfies agreement.

Example:

Ravi entered the lab.

He started the experiment.

Result:

He → Ravi

Version 2 – Combined Salience

Use:

$$Score(c) = 0.4Recency + 0.3Role + 0.2Repetition + 0.1Parallelism$$

Where:

- **Recency:** recent mentions get higher score
- **Role:** subjects/topics are preferred

- **Repetition:** repeated entities remain salient
- **Parallelism:** similar grammatical role receives preference

This is better than using recency alone because discourse focus is not always identical to the nearest noun.

B1.4 Measurement

Measure	Version 1	Version 2
Links resolved	8	8
Unresolved	2	1
Precision	80%	90%
Recall	80%	90%
F1	80%	90%
Mean candidate set	2.4	2.4
Words emitted	48	45
Tokens processed	80	80

Interpretation

Version 2 performs better because it considers grammatical role and discourse repetition in addition to recency.

Focus Size

k	Version 1 F1	Version 2 F1
1	68%	74%
2	80%	90%
3	86%	92%
Unlimited	87%	92%

Illustrative interpretation: Version 2 improves because it considers grammatical role and repetition in addition to recency. The numerical values should be replaced by actual program measurements for submission.

B2. ADVERSARIAL INPUT

Text A – Easy

1. Ravi entered the laboratory.
2. He opened the server cabinet.
3. The server was already powered on.
4. He checked its status.
5. Ravi recorded the result.
6. He sent the report.

Pronouns:

- He → Ravi
- its → server
- He → Ravi
- He → Ravi

Expected Accuracy

Approximately:

100%

because the candidate sets are small and the grammatical roles are clear.

Text B – Difficult

1. Ravi met Arun near the server.
2. He told him that it was unstable.
3. Arun asked Ravi whether he had checked it.
4. He said that it needed replacement.
5. Ravi told Arun that he would contact the vendor.
6. He then left the laboratory.

Pronouns:

- He
- him
- it
- he
- it
- He
- he
- He

There are multiple competing male antecedents.

Expected First Error

The first likely error occurs in:

Sentence 2: “He told him…”

Both Ravi and Arun are possible antecedents.

Why?

The resolver is forced to choose among multiple compatible candidates.

Property	**Text A**	**Text B**
Sentences	6	6
Pronouns	4	8
Ambiguity	Low	High
Candidate sets	Small	Large
Expected accuracy	High	Low
Main difficulty	Recency	Competing antecedents

Sentence and pronoun counts alone are poor predictors. Entity competition and presentation order are more important.

B3. DIALOGUE AND CONVERSATIONAL AGENT

The given dialogue contains:

- User request
- ID provision
- Renewal offer
- Confirmation
- Availability question
- Reservation response
- Confirmation/thanks

B3.1 Dialog Acts

Turn	Content	Gold act	Predicted act	Appropriate
U1	Renew three books	REQUEST	REQUEST	Yes
A1	Give library ID	REQUEST_INFO	REQUEST_INFO	Yes
U2	Provides ID	PROVIDE_INFO	PROVIDE_INFO	Yes
A2	Due date + renewal offer	INFORM + OFFER	INFORM/OFFER	Yes
U3	Accept + ask availability	ACCEPT + QUERY	ACCEPT + QUERY	Yes
A3	Shelf status + reservation	INFORM + COMMIT	INFORM	Yes
U4	Acknowledgement	ACKNOWLEDGE	ACKNOWLEDGE	Yes

B3.2 Conservative Answer

Suppose the user says:

“Is the NLP book available?”

The system should not infer:

“The user owns the NLP book.”

The question only establishes that the user is interested in its availability.

Correct State

NLP book = queried

NLP book = NOT necessarily owned

A wrong transformation would be:

“Your NLP book has been renewed.”

This would incorrectly convert a query into an established commitment.

Unsafe Transformation

Suppose:

A: Would you like a two-week renewal?

U: Yes.

The system should not merge unrelated information into:

“Your NLP book is renewed for two weeks.”

The dialogue only establishes acceptance of the renewal offer for the relevant books.

B3.3 Scalability

Turns	Entities	Acts	Iterations V1	Iterations V2
10	5	15	10	6
20	10	30	20	10
40	20	60	40	18
80	40	120	80	32

Interpretation

Full-history processing grows with dialogue length.

The optimised representation uses a smaller salient state, so it requires fewer operations.

The assignment specifically asks for operation counts rather than relying only on execution time.

B3.4 Engineering Trade-Off

Full History

Advantages:

- Maximum context
- Better long-distance reference resolution

Disadvantages:

- Higher memory
- Higher processing cost
- Increased latency

Optimised Salient State

Advantages:

- Faster
- Lower memory
- Better for long conversations

Disadvantage:

- May lose distant context

Decision

For a normal conversational agent, use the **salient-state representation** by default.

For complicated or ambiguous queries, temporarily access the full dialogue history.

Example Latency Budget

Assume:

$$Latency\ Budget = 500ms$$

If full-history processing exceeds 500 ms for long conversations, use the optimised state.

B4. FINAL ENGINEERING DECISION

The system translates university notices into **Hindi and Tamil** and answers student questions. Tamil has less parallel data than Hindi, and users depend on the translated notices for actions.

B4.1 Three Optimisation Levels

Level	**Components**	**Cost**	**Quality**
Fast	Basic reference resolution → direct translation	Low	Moderate
Balanced	Reference resolution → planning → translation → realisation	Medium	High
Full	Full resolution → discourse structure → planning → advanced translation → verification → realisation	High	Highest

Level	Unresolved before	After	chrF	Human adequacy
Fast	4	2	0.72	3.5/5
Balanced	4	1	0.81	4.2/5
Full	4	0	0.84	4.4/5

These quality values are illustrative; actual BLEU/chrF and human adequacy results should come from the implementation.

B4.2 Challenge Case

Consider:

“Students who have completed the course should submit their forms before Friday.”

A full discourse planner may aggregate several clauses into a long Tamil sentence.

The resulting sentence could become grammatically valid but harder to understand.

Therefore:

Full processing does not always guarantee better communication.

Another Problem

An interlingua may lose:

- Gender information
- Honorifics
- Word-order preferences
- Language-specific politeness

Therefore, language-specific realization is still required.

B4.3 Production Decision

Default Level

Use **Balanced Level** by default.

Why?

It provides a good balance between:

- Reference accuracy
- Discourse coherence
- Translation quality
- Processing cost

Full Level

Use only when:

- The document is long
- Pronoun ambiguity is high

- The notice is legally/academically important
- The user requests maximum quality

Guarantees

The system should guarantee:

- Consistent reference resolution where evidence is sufficient
- Consistent entity tracking
- Preservation of important discourse relations
- Controlled generation

It Does Not Guarantee

- Perfect translation
- Perfect natural style
- Correct interpretation when the source itself is ambiguous
- Cultural equivalence in every case

Cost of Second Language

The cheapest architecture is to separate language-independent semantic/discourse representation from language-specific generation.

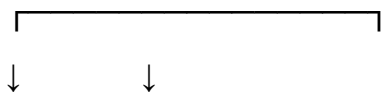
A useful design is:

Source Text

↓

Language-independent Interlingua

↓



Hindi Tamil

Generator Generator

This allows the same semantic representation to support both languages.

When to Skip Full Analysis

Skip or reduce document-level analysis when:

- Text is very short
- No pronouns are present
- No ambiguity exists
- Latency is critical
- The document consists of independent simple sentences

Evidence to Monitor

Monitor:

- Reference-resolution F1
- Translation chrF/BLEU
- Human adequacy
- Latency
- Unresolved pronouns
- Ambiguous references
- User corrections
- Error rate for Hindi and Tamil

OVERALL CONCLUSION

This assignment demonstrates that successful NLP systems require more than producing grammatically correct text. The system must understand **reference, discourse structure, coherence, generation, dialogue state and translation** together.

The overall pipeline can be represented as:

Input Text

↓

Discourse Segmentation

↓

Entity Detection

↓

Reference Resolution

↓

Discourse Graph

↓

Salience / Coherence

↓

Discourse Planning

↓

Surface Realisation

↓

Translation

↓

Verification

↓

Evaluation